

Object-Oriented Relationships in C++

Association • Aggregation • Composition • UML Diagrams

Understanding how objects interact in real-world systems.
(Pre-Final Project Concept Refresher)



Quick Recap: OOP Concepts

- **Classes / Objects** → encapsulate data & behavior
- **Inheritance** → “is-a” hierarchy (reuse & extension)
- **Polymorphism** → flexible interfaces
- **Virtual functions** → enable runtime behavior changes
- **Why Class Relationships Matter**
 - Real-world systems = multiple objects interacting
 - Helps identify ownership and lifetime dependencies
 - Crucial for **software architecture & UML modeling**
 - Makes your final project designs more modular and maintainable

Object-Oriented Relationships in C++

Association • Aggregation • Composition

Association Relationship in Classes

- A *general relationship* between classes — both can exist independently.
 - A Patient is *associated* with a Doctor.
 - They can exist independently — deleting one doesn't destroy the other.
 - Relationship often implemented via **pointers** or **references**.
- **UML Notation:**
 - Solid line with arrow (optional direction)
 - *A Patient is treated by a Doctor.*

```
class Doctor {  
public:  
    std::string name;  
    void treat(Patient* p);  
};  
  
class Patient {  
public:  
    void assignDoctor(Doctor* d) { doctor = d; }  
private:  
    Doctor* doctor; // Association (link, not ownership)  
};
```

Aggregation Relationship in Classes

- A “*has-a*” relationship where the container holds references to independent objects.
 - Department aggregates Course objects.
 - Courses can exist even if the Department is deleted (no ownership).
 - Common in manager–component or controller–model setups.
- **UML Notation:**
 - White diamond at the “whole” end
 - *Department* o— *Course*

```
class Course {
public:
    std::string name;
};

class Department {
public:
    void addCourse(Course* c) { courses.push_back(c); }
private:
    std::vector<Course*> courses; // Aggregation
};
```

Composition Relationship in Classes

- A *strong ownership* relationship — parts can't exist without the whole.
 - Car *owns* its Engine.
 - When the Car is destroyed, the Engine is too.
 - Represents *part-whole* dependency.
- **UML Notation:**
 - Black diamond at the “whole” end
 - Car  Engine

```
class Engine {
public:
    Engine(int hp): horsepower(hp) {}
    void start() { std::cout << "Engine running\n"; }
private:
    int horsepower;
};

class Car {
public:
    Car(std::string brand): brand(brand), engine(250) {}
    void drive() { engine.start(); }
private:
    std::string brand;
    Engine engine; // Composition
};
```

UML Relationship Summary

Relationship	Lifetime Dependency	UML Symbol	Example
Association	None	Simple line	Doctor–Patient
Aggregation	Independent	○ (white diamond)	Department–Course
Composition	Dependent	◆ (black diamond)	Car–Engine

UML Class Diagram Overview

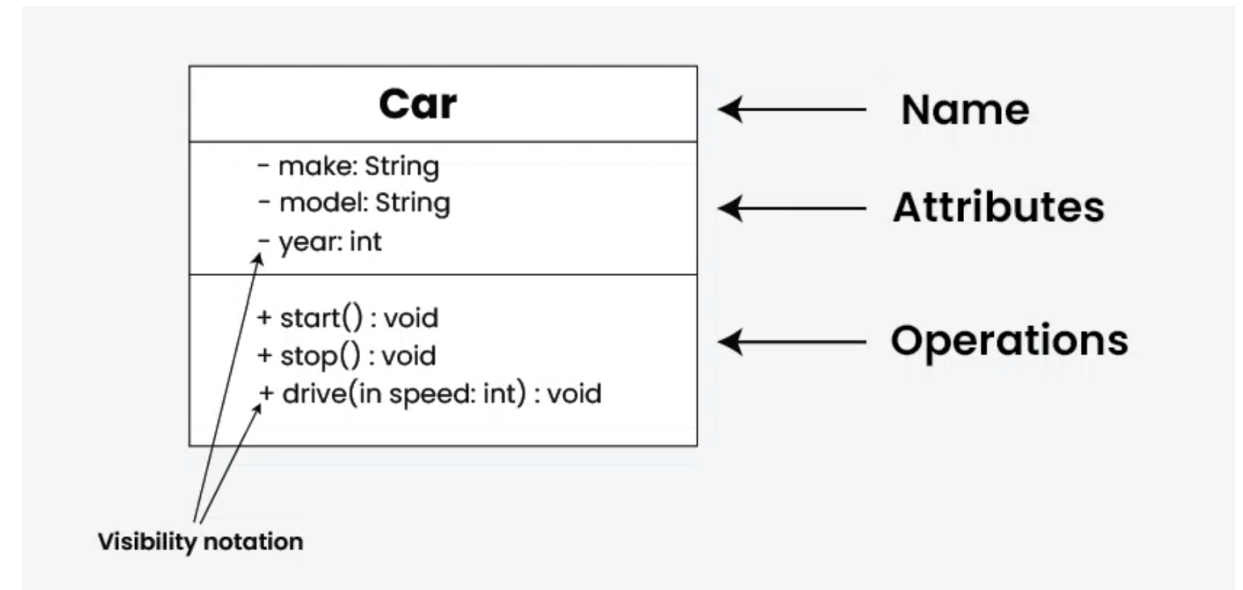


What is a UML Class Diagram?

- UML class diagram visually represents the **structure** of a system by showing:
- **Classes**, their **attributes** and **methods**
- **Relationships** between classes
- **Why it matters:**
 - Helps everyone (developers, designers, PMs) understand how the system is organized
 - Clarifies how components **interact**
 - Acts as documentation of system structure
 - *Think of it as a "blueprint" for your software system.*

UML Class Diagram

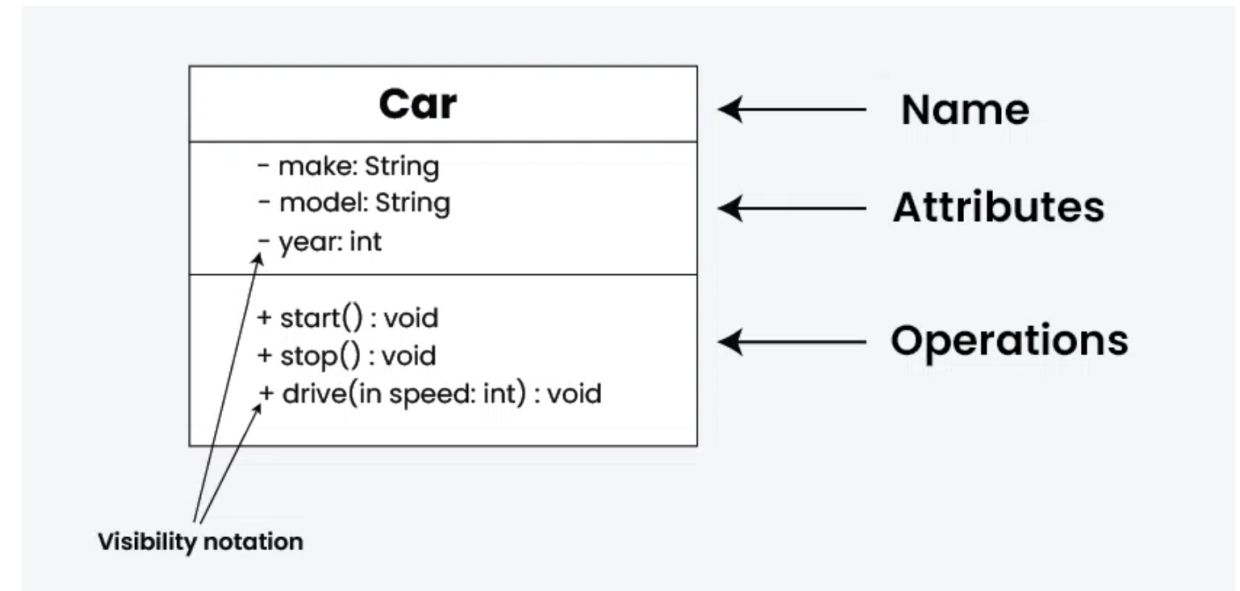
- **UML Class Notation**
- **Classes are drawn as boxes with three compartments:**
- **Top:** *Class Name* (centered, bold)
- **Middle:** *Attributes* (data members)
- **Bottom:** *Methods* (functions or operations)



UML Class Diagram

• Visibility notations

Symbol	Meaning
+	Public
-	Private
#	Protected
~	Package (default)



UML Class Diagram

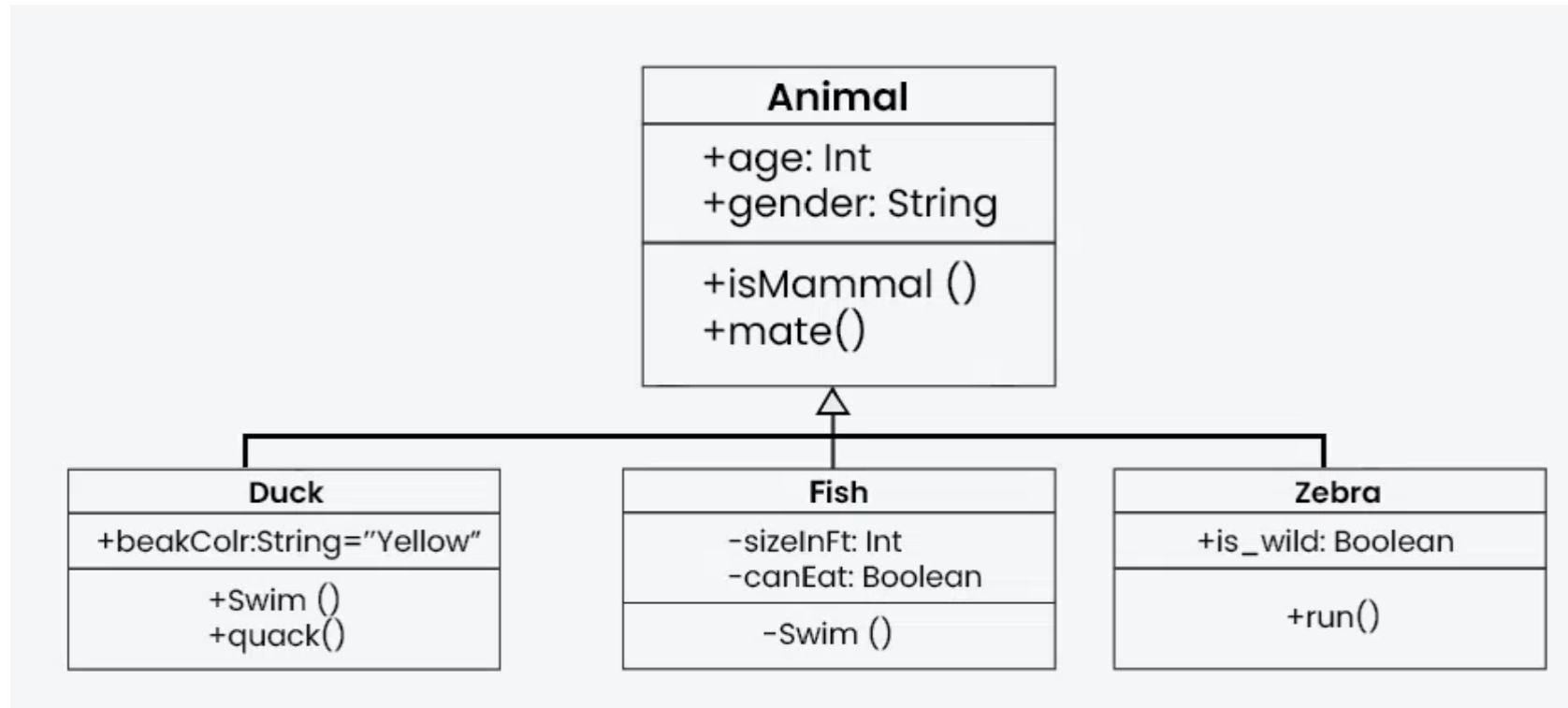
- Parameter direction shows the **flow of data** between classes during method calls.
- This helps clarify how data moves across classes.
- Example: `+printDocument(in fileName: String, out status: Boolean)`

Direction	Description	UML Symbol
<code>in</code>	Input to a method	→ toward the class
<code>out</code>	Output from a method	← away from the class
<code>inout</code>	Both input and output	↔ both directions

Relationships Between Classes

Type	Description	UML Symbol
Association	Basic connection between two classes	Solid line
Directed Association	Connection with a defined direction	Solid line with arrow
Aggregation	"Whole-Part" (parts exist independently)	○ White diamond
Composition	"Whole-Part" (parts depend on the whole)	◆ Black diamond
Generalization	Inheritance ("is-a")	Hollow arrow
Dependency / Usage	Class uses another class temporarily	Dashed arrow line

UML Diagram Example





Benefits of UML Diagrams

- **Purpose:**

- Clearly depicts OOP concepts: inheritance, abstraction, relationships
- Provides foundation for deployment and component diagrams
- Supports **forward and reverse engineering**

- **Benefits:**

- Visualizes software architecture
- Clarifies class responsibilities and connections
- Improves communication between technical and non-technical team members
- Guides coding and ensures design–implementation alignment
- Enables automated **code generation** in many IDEs



SFML quick tutorial



SFML (Simple and Fast Multimedia Library)

- A modern C++ library for **graphics, games, and interactive apps**. Think of it as a simpler way to do what OpenGL or SDL can — but with a clean, object-oriented design.
- **Core Modules**
 - `sf::System` – Time, threads, basic utilities
 - `sf::Window` – Event handling, window creation
 - `sf::Graphics` – Shapes, text, textures, sprites
 - `sf::Audio` – Sound and music
 - `sf::Network` – Networking (optional)
- **Why It Fits This Course**
 - Uses **classes, objects, inheritance** → reinforces OOP.
 - Lightweight, cross-platform, and easy to install.
 - Great for **hands-on visual learning** — see your C++ code in action!
 - Opens the door to **game loops, events, and animations**.



How to Navigate SFML

- [Tutorials \(SFML / Learn\)](http://www.sfml-dev.org/tutorials/2.6/) (<http://www.sfml-dev.org/tutorials/2.6/>)
- SFML is well-structured and object-oriented — every concept is a **class**, and each class is part of one of five **modules**:
system, window, graphics, audio, network.
- **Know the Class You Need**
 - Want to **draw something**? → Go to **Graphics Module**
 - Want to **detect input or close windows**? → Go to **Window Module**
 - Want to **play sound**? → Go to **Audio Module**
- Example let's draw a circle shape:
 - `sf::CircleShape`
 - inherits from `sf::Shape` → which inherits from `sf::Drawable`
 - `sf::Drawable` can be drawn with `window.draw(...)`
- Reinforces **class hierarchy navigation** + OOP understanding.



How to Navigate SFML

- **se the Official Tutorials (Before the API Docs)**
- The SFML tutorials are **the fastest way** to get productive:
<https://www.sfml-dev.org/tutorials/2.6/>
- Recommend starting with:
 - **Graphics → Drawing shapes**
 - **Window → Events and input**
 - **System → Time and clock (for animations)**
- Tip: Don't memorize, browse intentionally. Each tutorial has **short examples** that compile directly



How to Navigate SFML

- **se the Official Tutorials (Before the API Docs)**
- The SFML tutorials are **the fastest way** to get productive:
<https://www.sfml-dev.org/tutorials/2.6/>
- Recommend starting with:
 - **Graphics → Drawing shapes**
 - **Window → Events and input**
 - **System → Time and clock (for animations)**
- Tip: Don't memorize, browse intentionally. Each tutorial has **short examples** that compile directly



Agenda for today!

- Recap from last couple of classes!
- Continue Object Relationships in C++
- Study example UML Diagram in detail
- Team class activity on UML Diagrams!
- [UML Class Diagrams Youtube Video](#)



Recap – Virtual Functions

- Virtual functions enable **runtime decision-making**—the correct function is called based on the **object's actual type**, not the pointer/reference type.
- **Without virtual:**
 - C++ uses **static binding** (decides at compile time).
 - The base class function executes, even if the pointer points to a derived object.
- **With virtual:**
 - C++ uses **dynamic binding** (decides at runtime).
 - Calls the derived class's version automatically.
- **Key Takeaway:**
 - Use virtual for base functions that may be redefined, and override in derived classes for safety and clarity.

```
class Base {  
public:  
    virtual void print();    // Virtual  
};  
class Derived : public Base {  
public:  
    void print() override;    // Override keyword (good practice)  
};
```



Recap – Polymorphism

- Polymorphism allows a single interface (like a base pointer) to represent **multiple derived types** and call the correct function automatically.
- Without Polymorphism: **Slicing Problem**
- **Why it matters:**
 - Enables **flexible and extensible** designs.
 - Supports storing **heterogeneous objects** in collections (Question* quiz[]).
 - Core to modern frameworks, UI, and game engines.
- **Key Takeaways:**
 - Use **pointers or references** (not direct objects) to preserve polymorphism.
 - Always make **base class destructors virtual** to ensure correct cleanup.

```
class Question {
public:
    virtual void display() const;
};

class ChoiceQuestion : public Question {
public:
    void display() const override;
};

int main(){
    Question* quiz[2] = {new Question(), new ChoiceQuestion()};
    quiz[1]->display();    // Calls ChoiceQuestion::display()
}
```



Recap – Pure Virtual Functions and Abstract Classes

- **Pure Virtual Function:**

- A function declared with = 0 — no implementation in the base class.

- **Abstract Class:**

- Contains ≥ 1 pure virtual function.
- **Cannot be instantiated** directly.
- Derived classes must override pure virtual functions to be concrete.

- **Benefits:**

- Defines a **common interface**.
- Enforces **implementation in derived classes**.
- Encourages **code reusability** and clean architecture.

```
class Shape {  
public:  
    virtual void draw() = 0;    // Pure virtual  
};
```



Recap from last time – Object Relationships and UML Class Diagrams

- Objects in OOP interact through **relationships**:
 - **Inheritance**: “is-a” (e.g. Vehicle – Car)
 - **Association**: “works-with” (e.g., *Patient–Doctor*).
 - **Aggregation**: “has-a” (e.g., *Department–Course*).
 - **Composition**: “owns” (e.g., *Car–Engine*).
- UML Class Diagrams visualize **structure** and **relationships**.
- Classes are represented as boxes with compartments
- Relationships are represented as lines with connectors i.e. circle, hollow circles, diamond, etc.
- Visibility notation recap:
 - + public, - private, # protected.

Recap from last time – Object Relationships

- **Association**

```
class Doctor {};  
class Patient { public: Doctor* doctor; };
```

- **Aggregation**

```
class Course {};  
class Department { public: vector<Course*>  
courses; };
```

- **Composition**

```
class Engine {};  
class Car { Engine engine; };
```

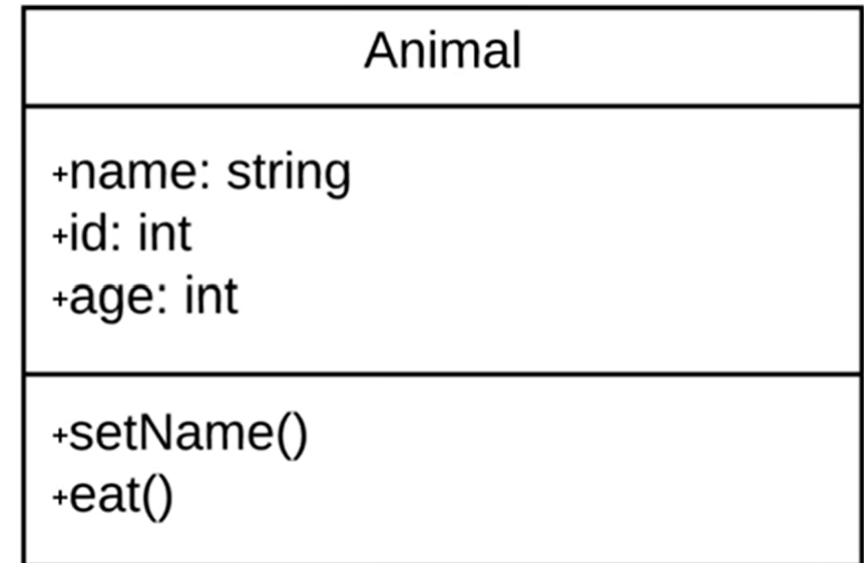


Recap from last time – Object Relationships

- **Patient — Doctor** denotes **Association**
- **Department ◇— Course** denotes **Aggregation**
- **Car ◆— Engine** denotes **Composition**

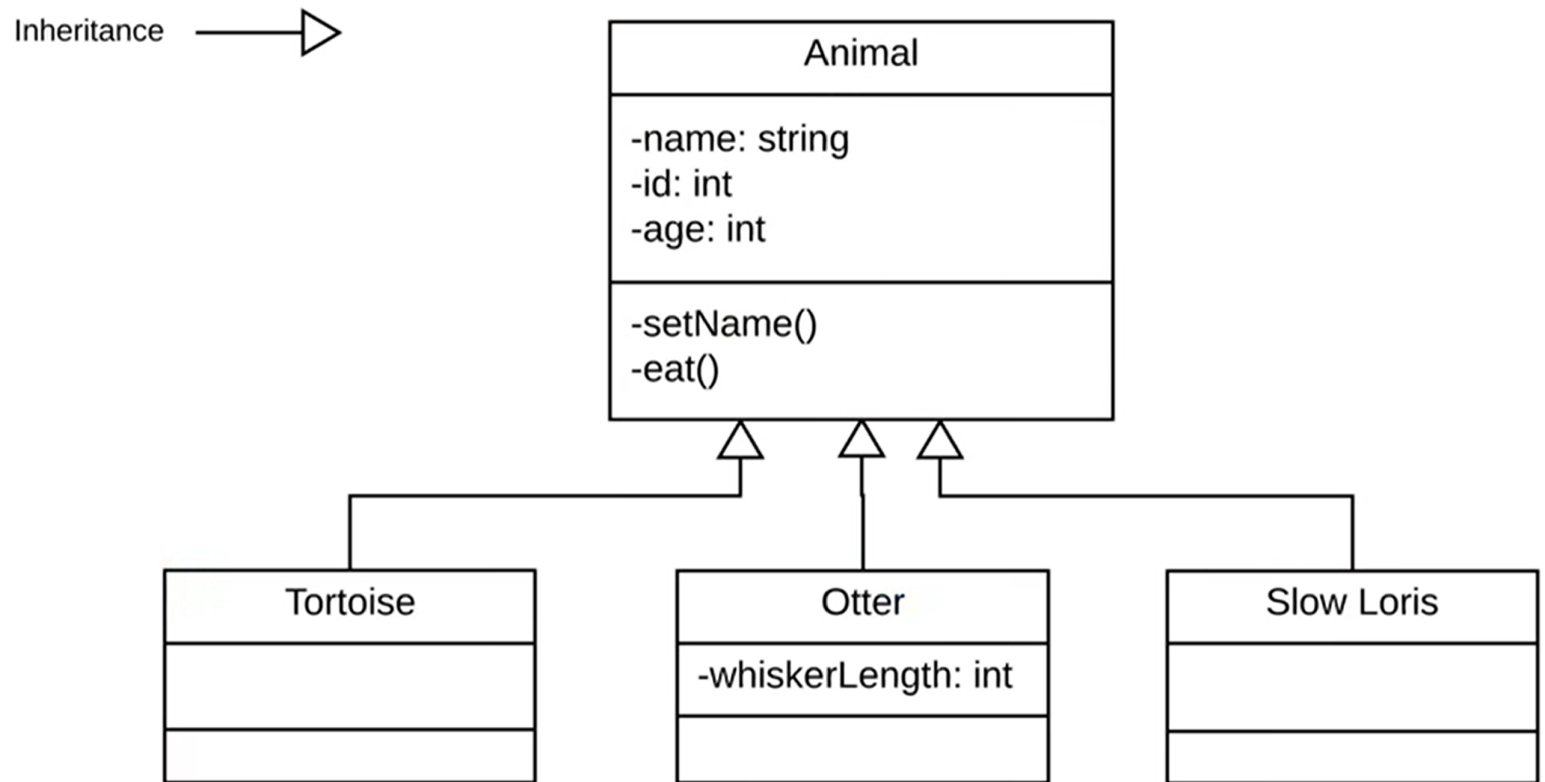
Recap – UML Class Representation

- Each class is shown as a **rectangle with three compartments**:
 - **Class name**
 - **Attributes (data members)**
 - **Methods (member functions)**
- Include **visibility** symbols:
 - + public
 - - private
 - # protected



UML Example Inheritance – Continued

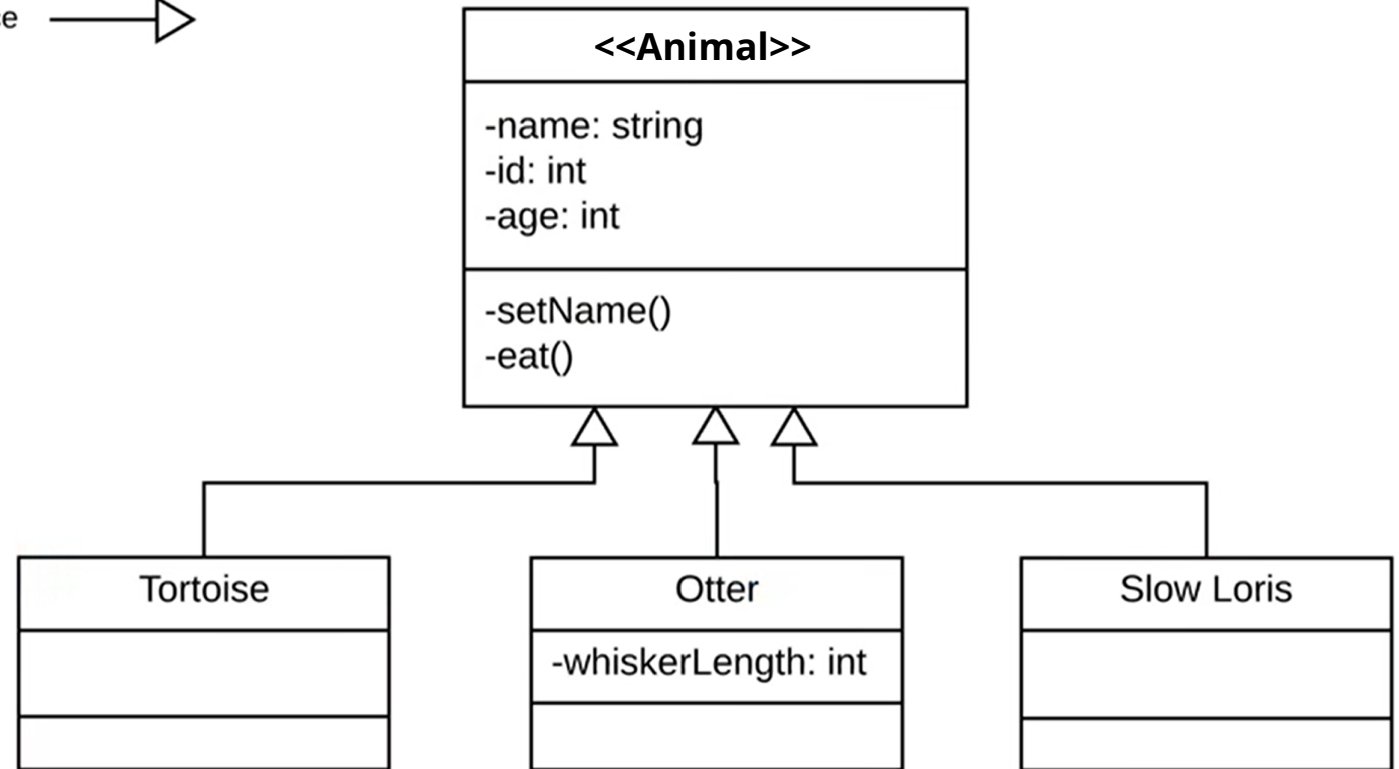
- Inheritance relationship is shown as a **hollow arrow**
- For attributes that are shared, only **need to define once in the base class** and don't need to declare them again in derived classes.



UML Example Inheritance – Continued

- **Abstract classes:** italicized name or <<Abstract>>.
- **Reminder:** Abstract Classes are kind of like templates which have at least one pure virtual function.
- `virtual void get_name()=0;`

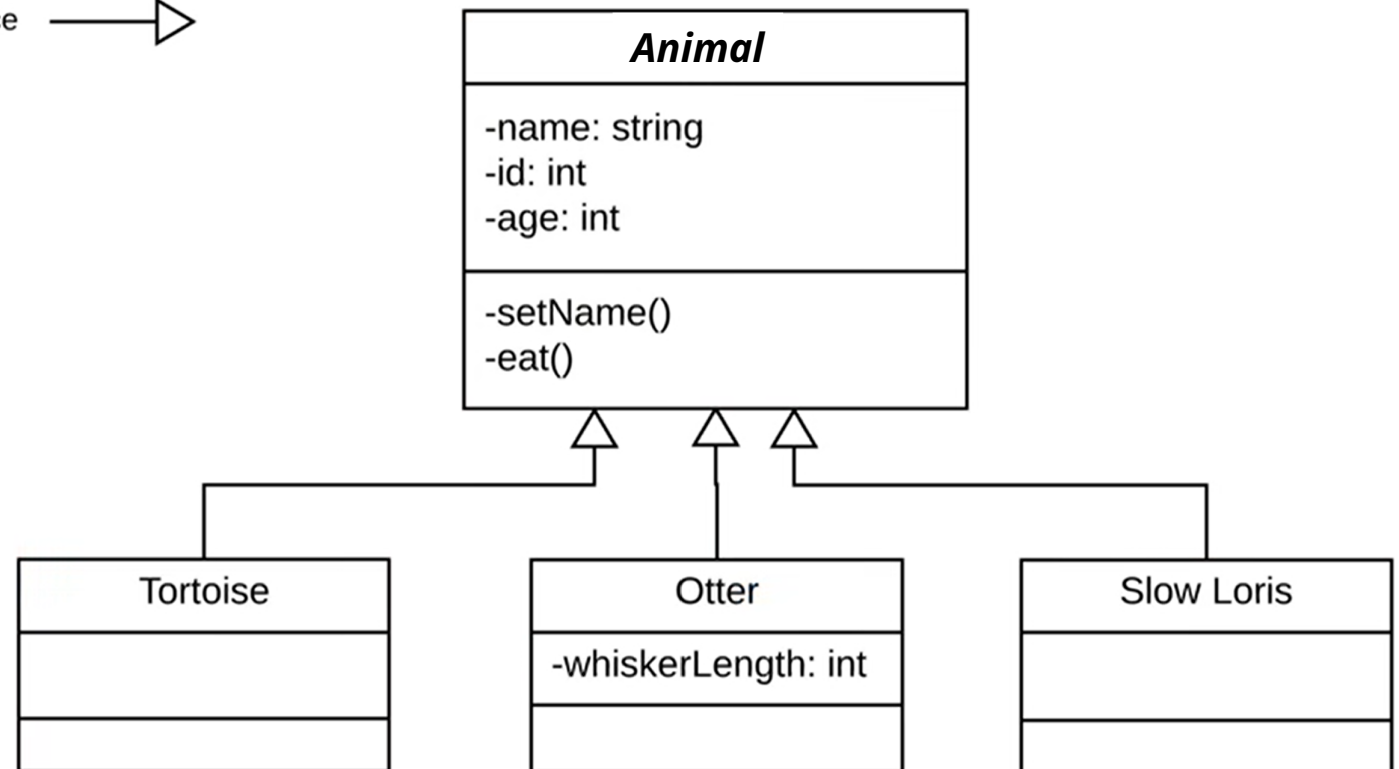
Inheritance →



UML Example Inheritance – Continued

- **Abstract classes:** italicized name or <<Abstract>>.
- **Reminder:** Abstract Classes are kind of like templates which have at least one pure virtual function.
- `virtual void get_name()=0;`

Inheritance →

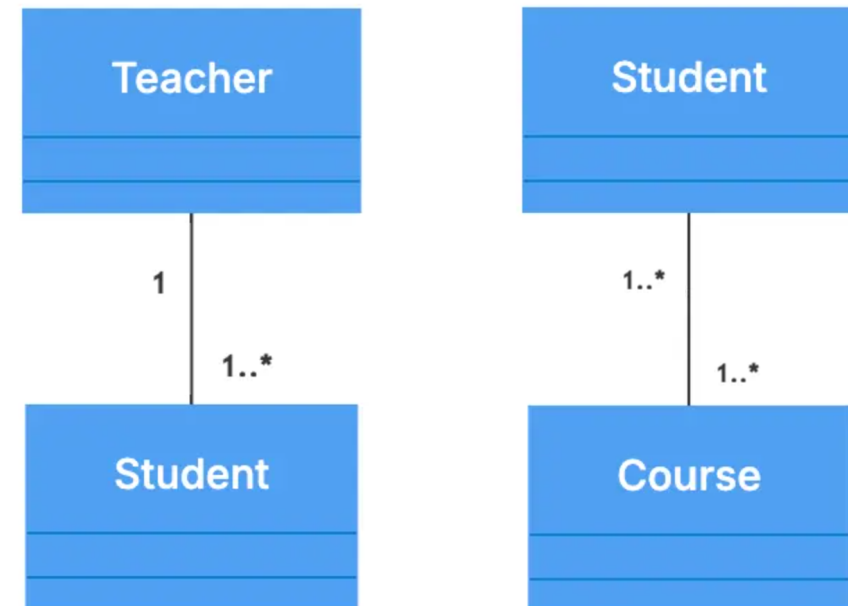


Multiplicity (Cardinality)

- Defines **how many instances** of one class **relate to another**. In other words set numerical constraints on the relationship!
- Syntax examples:
 - 1 → exactly one
 - 0..1 → optional
 - 0..* → Zero or Many
 - 1..* → One or Many
 - 1..5 → between 1 and 5
- These are usually read from both sides of the relationship.
- **Example: Customer 1..2 — 1..5 BankAccount**
 - One Customer can have 1-5 BankAccounts.
 - One BankAccount can belong to 1-2 Customers.
- So every time you see multiplicities on a UML Class Diagram, break it down into two parts.

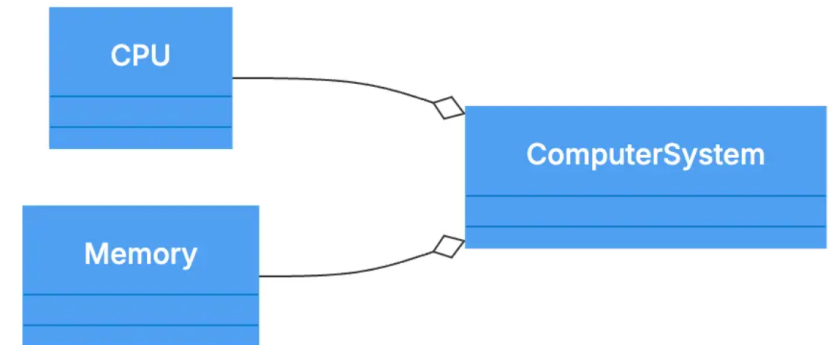
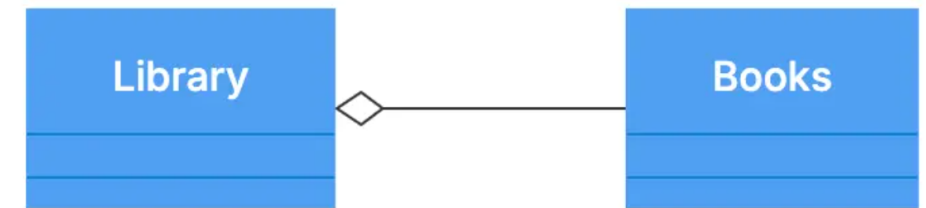
Multiplicity (Cardinality)

- **Examples:**
- **Teacher 1 — 1.. Student**
 - One to Many Relationship. A teacher can teach multiple students.
- **Student 1.. — 1.. Course**
 - Many to Many Relationship. A student can take multiple courses.



Object Relationships – Aggregation

- "has-a" relationship
- **Characteristics:**
 - **Weak Relationship:** The life cycle of the parts is not tightly coupled to the life cycle of the aggregate.
 - **Binary Relationship:** Aggregation typically involves two classes where one class is a container, and the other class is contained.
- **Examples :**
 - **Library and Books:** The library (whole) contains books (parts), but the books are not dependent on the library for their existence
 - **Computer System and CPU + Memory:**





Object Relationships – Aggregation

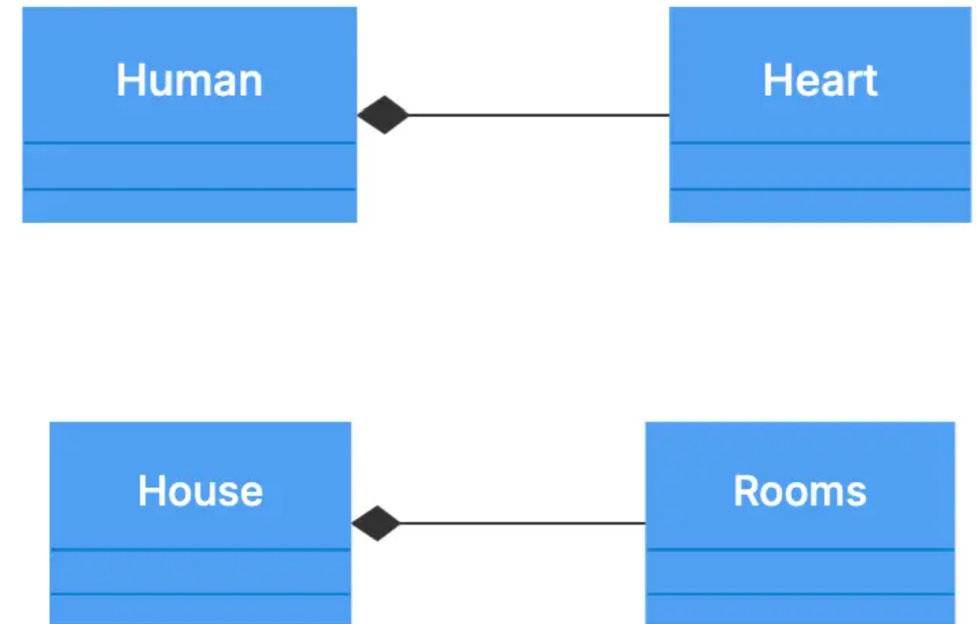
- Usually Implemented using **pointers** or **references to existing objects**
- **Person o— Address**

```
class Employee {  
    // Class members and methods  
};
```

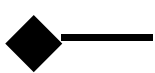
```
class Department {  
    std::vector<Employee*> employees;  
    // Class members and methods  
};
```

Object Relationships – Composition

- “part-whole” relationship
- **Characteristics:**
 - **Strong Ownership:** The lifecycles of the parts are tightly coupled with the lifecycle of the whole.
 - **Exclusive Containment:** Each part is contained in only one whole at a time and cannot be shared among multiple wholes.
- **Examples :**
 - **Human and Heart**
 - **House and Rooms**



Object Relationships – Composition

- Usually Implemented using **object initialization within class**
- Car  Engine

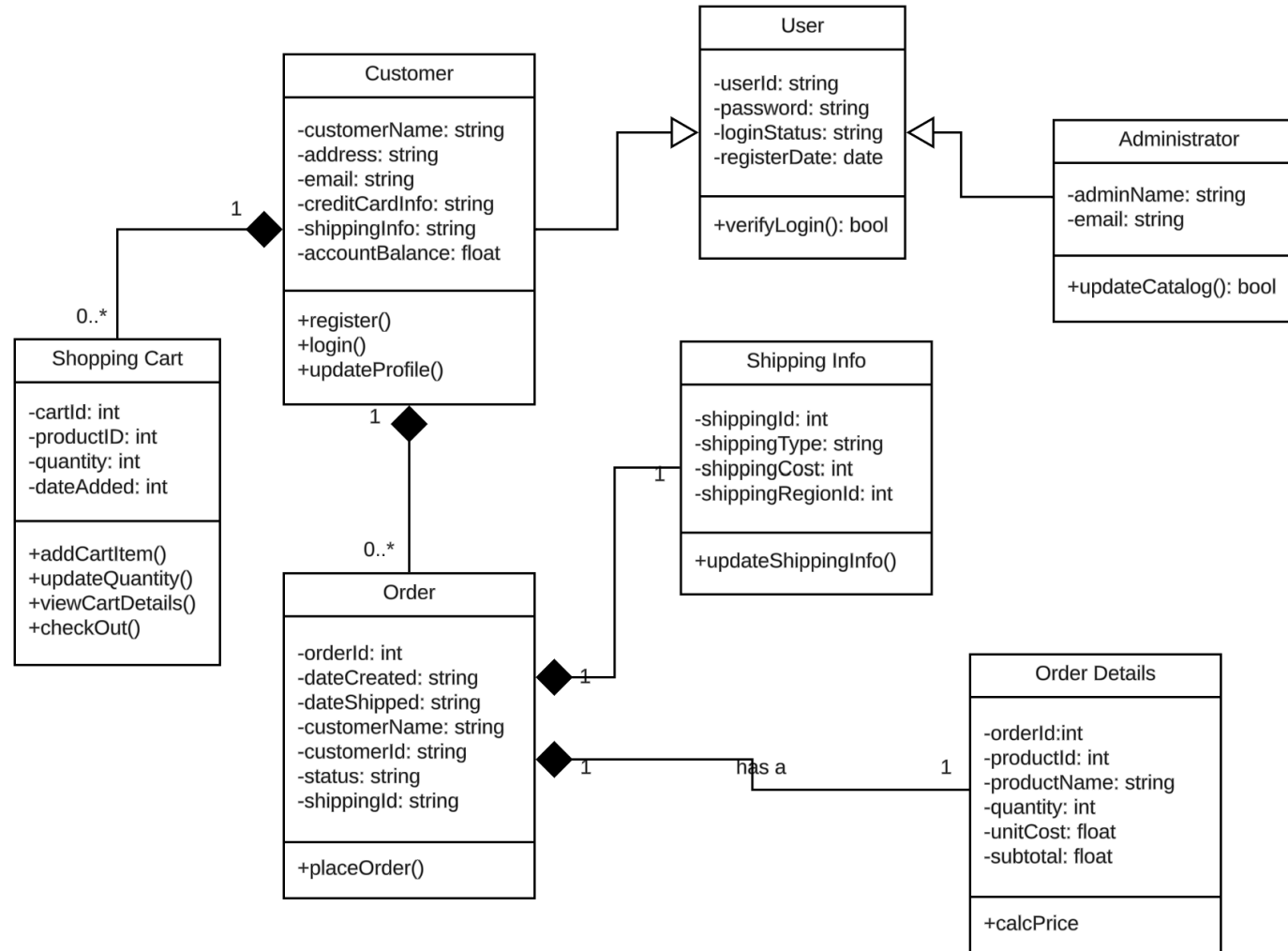
```
class Engine {  
    // Class members and methods  
};
```

```
class Car {  
    Engine engine;  
    // Class members and methods  
};
```

Summary of Object Relationships!

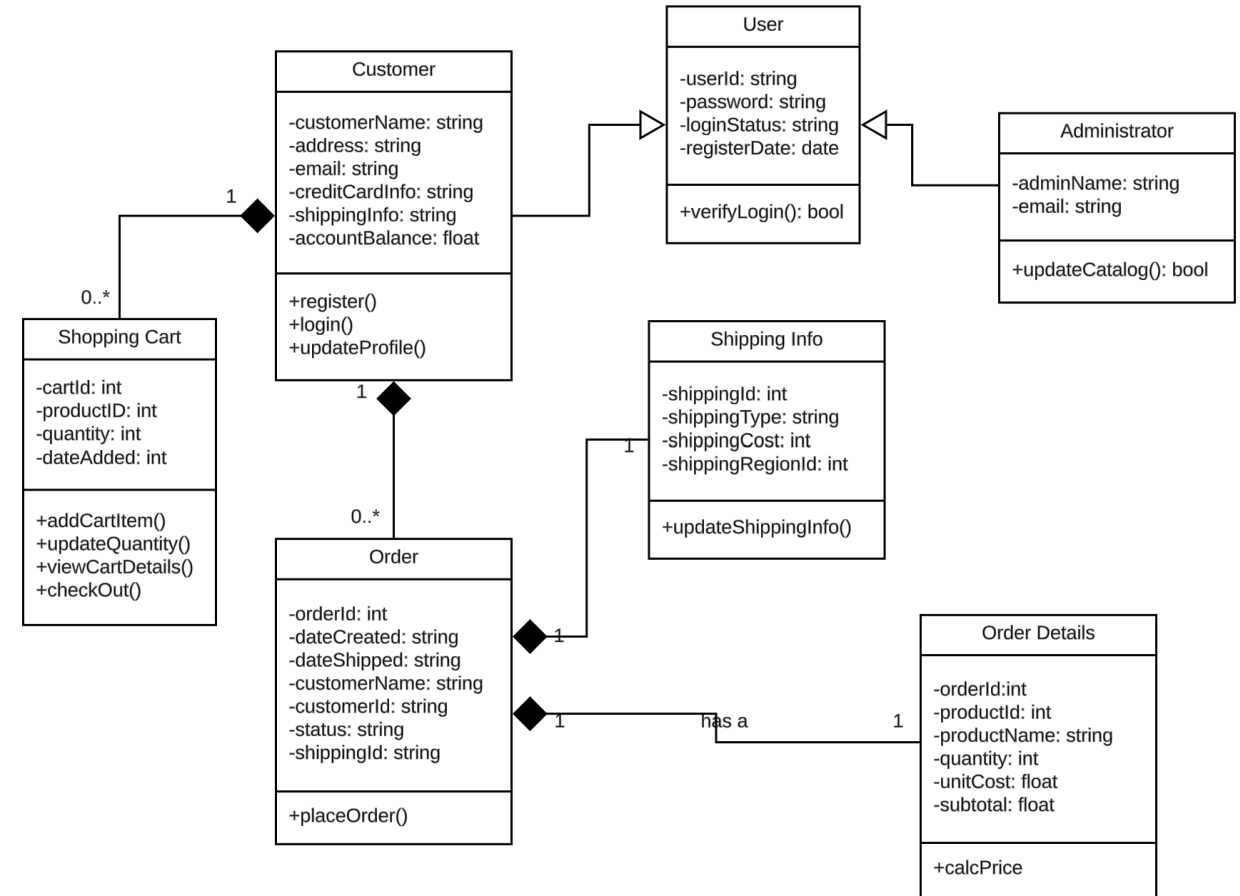
Property	Composition	Aggregation	Association
Relationship type	Whole/part	Whole/part	Otherwise unrelated
Members can belong to multiple classes	No	Yes	Yes
Members' existence managed by class	Yes	No	No
Directionality	Unidirectional	Unidirectional	Unidirectional or bidirectional
Relationship verb	Part-of	Has-a	Uses-a

An example UML Diagram



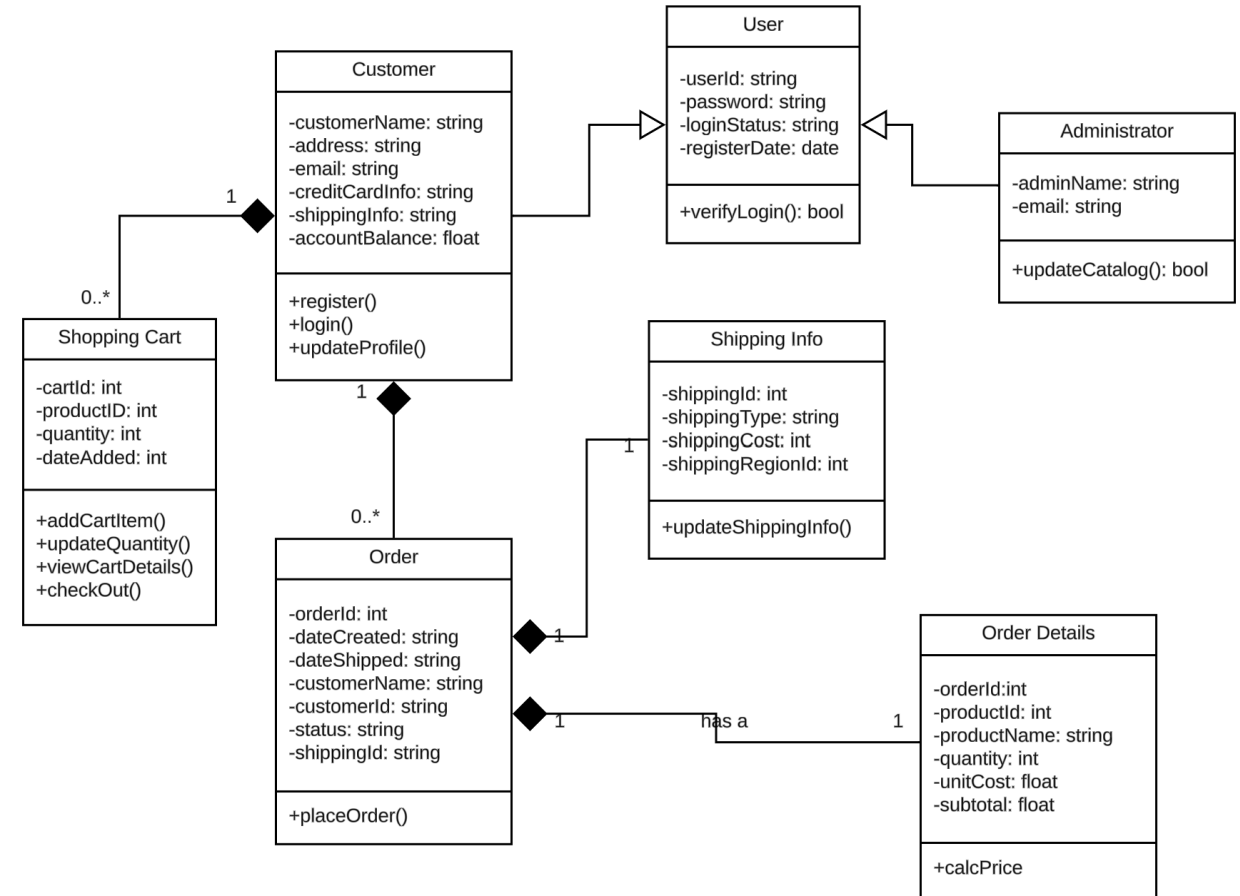
An example UML Diagram

- The system models **inheritance** using UML's open-triangle arrow: Customer and Administrator **inherit from** User.
- **User** defines shared attributes and operations common to all roles.
- **Customer** extends **User** with profile & payment details.
- **Administrator** extends User with catalog-management privileges.



An example UML Diagram

- **Composition** (black diamond):
 - A Customer owns one or more **ShoppingCart** and Order objects.
 - If the Customer is deleted, their **carts & orders** also disappear.
- **Multiplicity**:
 - One Customer → 0 to many Orders / Carts
 - One Cart belongs to exactly 1 Customer



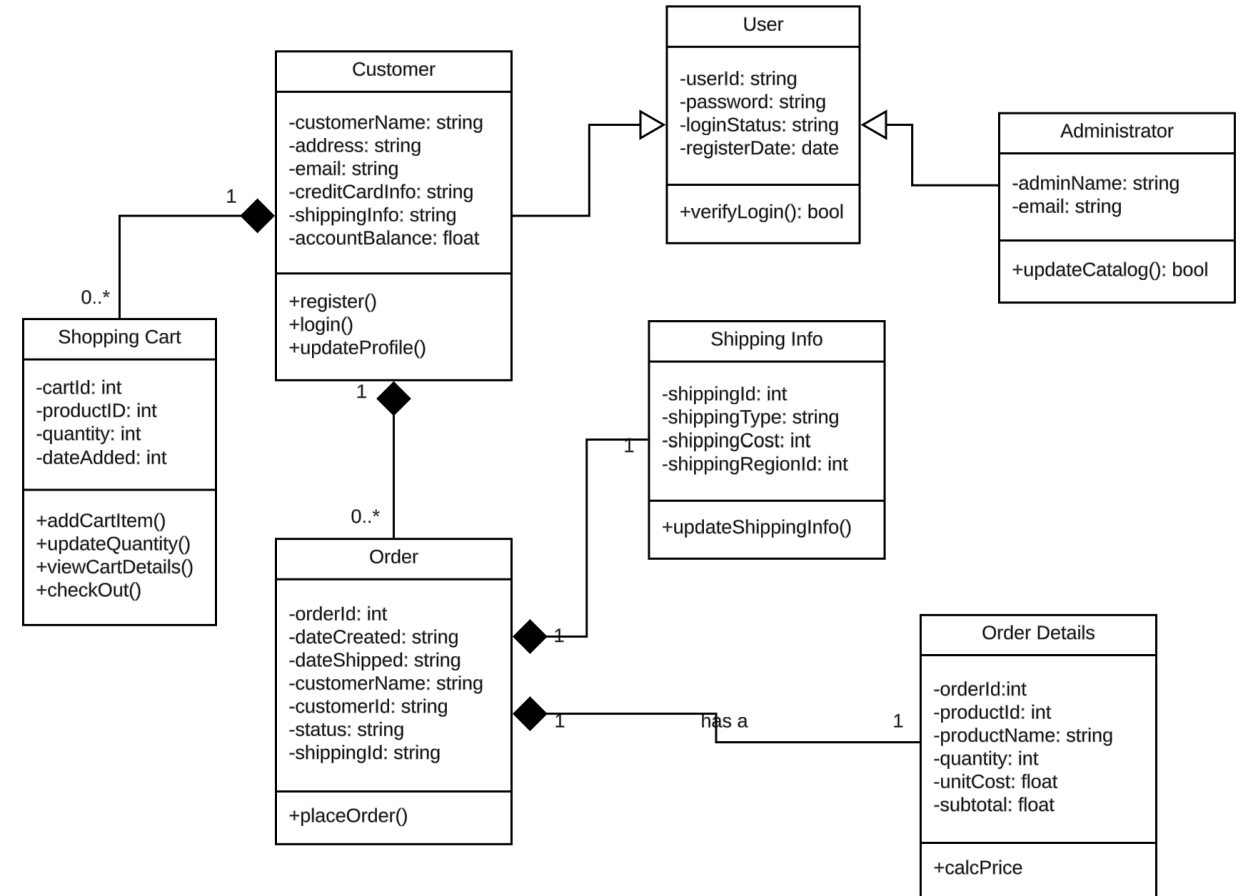
An example UML Diagram

- **Composition:**

- Each **Order** is composed of one **OrderDetails** as well as **Shipping Info**(black diamond).
- **Deleting** an order **deletes** its line items.

- **Multiplicity:**

- Order 1 — 1 OrderDetails
- Order 1 — 1 ShippingInfo





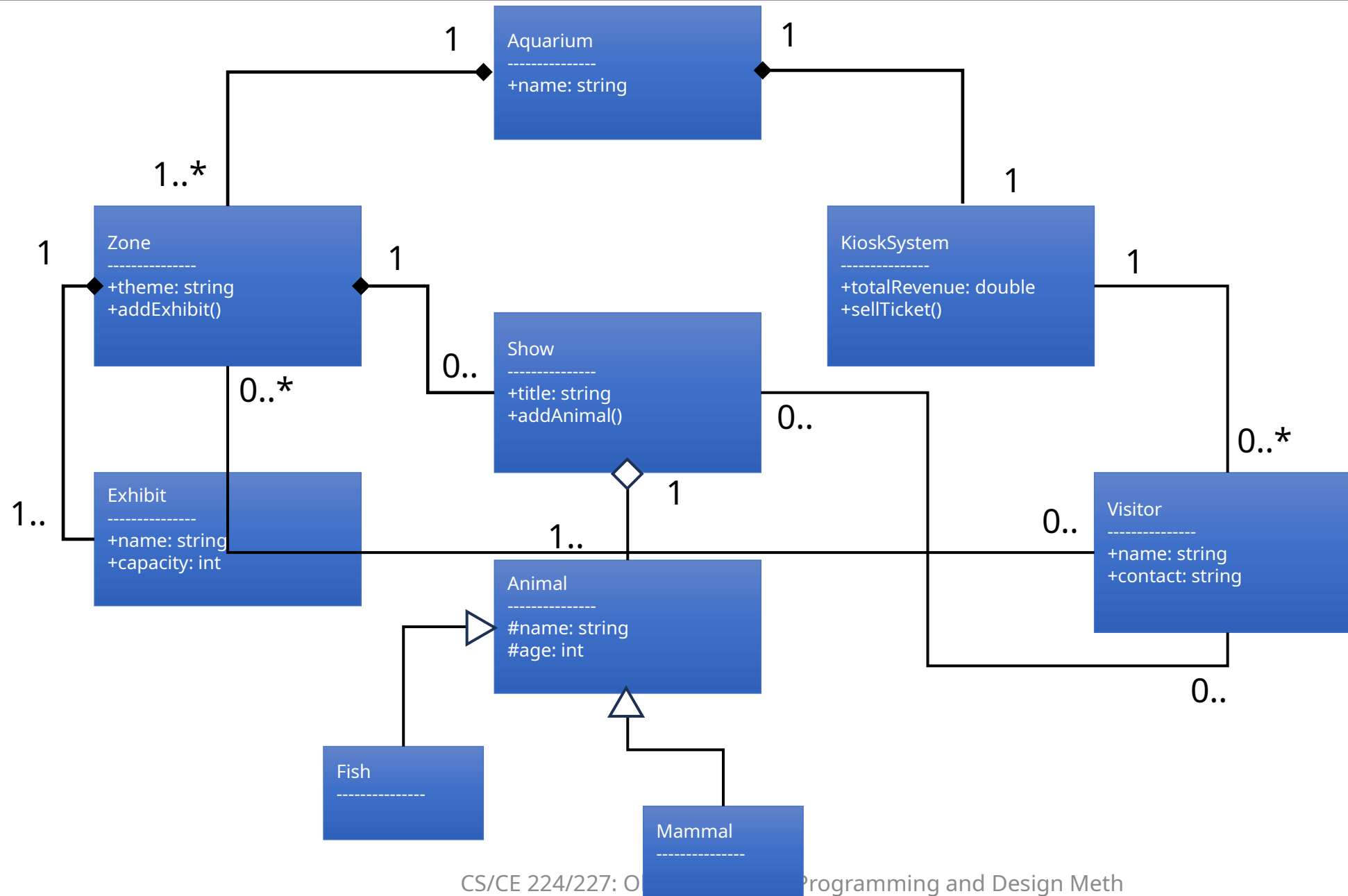
UML Diagram Team Class Activity

- Two Parts
 - Create a UML Diagram from the prompt in the next slide
 - Create a Skeleton in C++ with each class and how you would implement it



UML Diagram Team Class Activity

- AquaRealm is a smart, tech-driven aquarium park with these key components:
- **Zones & Exhibits:**
 - Each zone (e.g., *Coral Kingdom*, *Predator Bay*) has themed exhibits like “Shark Tunnel” or “Clownfish Reef.”
 - Managed by the main **Aquarium**, with unique sound, lighting, and environmental controls.
- **Digital Kiosk System:**
 - Handles ticketing, show scheduling, and revenue tracking.
 - Supports both online and on-site bookings.
- **Shows & Animals:**
 - Live demonstrations such as “Feeding Time” or “Shark Dive.”
 - Each show features one or more **Animals** (e.g., Shark, Dolphin, Turtle).
 - **Animal** is a superclass with subclasses (*Fish*, *Mammal*).
- **Visitors:**
 - Can browse zones, book shows, and purchase tickets/snacks via the kiosk.
 - Each visitor may attend multiple shows.





Some other Resources to check out!

- [UML Class Diagrams Youtube Video](#)
- PDF on UML Diagrams uploaded to Canvas!